

CS102L Data Structures & Algorithms

Lab 9B



DEPARTMENT OF COMPUTER SCIENCE
DHANANI SCHOOL OF SCIENCE AND ENGINEERING
HABIB UNIVERSITY
SPRING 2026

Contents

1	Introduction	2
1.1	Instructions	2
1.2	Marking scheme	2
1.3	Lab Objectives	2
1.4	Late submission policy	3
1.5	Use of AI	3
1.6	Viva	3
1.7	Prerequisites: Dictionaries	3
2	Lab Exercises	4
2.1	Helper Functions	4
2.1.1	Testing	5
2.2	Question 1	5
2.2.1	Testing	6
2.3	Question 2	6
2.3.1	Testing	6
2.4	Question 3	7
2.4.1	Testing	7

Introduction

1.1 Instructions

- This lab will contribute 1% towards your final grade.
- We will try expanding on what we did in last lab on graphs and try to build more functions on it.
- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *lab9b-aa12345.zip* where *aa12345* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise of both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 100.

- 50 Marks are for the completion of the lab.
- 50 Marks are for progress and attendance during the lab.

1.3 Lab Objectives

- To learn how to create graphs in Python
- To learn how to manipulate graphs in Python
- To learn how to create graph applications using the different helper functions of graphs

1.4 Late submission policy

There is no late submission policy for the lab.

1.5 Use of AI

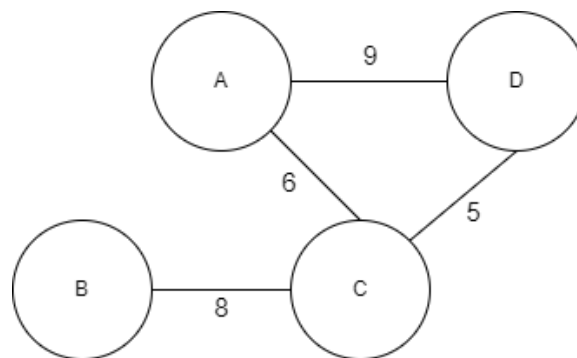
Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.6 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

1.7 Prerequisites: Dictionaries

For this lab, you will be working in Python Structure Dictionary. Please recap your concepts about dictionaries from CS101 as they will be required for this lab. Have a look at following undirected weighted graph:



A possible implementation of this Graph using dictionary is given below:

```
G= {  
  "A": [( "D" ,9) ,( "C" ,6)] ,  
  "B": [( "C" ,8)] ,  
  "C": [( "A" ,6) ,( "B" ,8) ,( "D" ,5)] ,  
  "D": [( "A" ,9) ,( "C" ,5)] ,  
}
```

Here, each Key is representing a node and each value is a list of tuples where each tuple is storing the node and weight to which the node in Key is connected to.

Lab Exercises

2.1 Helper Functions

These functions should be implemented in `helper_functions.py`.

Helper Function are really important for working on graphs. These function should be implemented in `helper_functions.py`.

- **in_out_degree(G):** This function will take a directed graph G as a parameter and return In Degree and Out Degree for each node in the Graph. This function will make a dictionary where each node will be the key and its value will be a tuple which will contain in degree and out degree respectively and return it.
- **degree(G):** This function will take an undirected graph G as a parameter and return the degree for each node in the Graph.
- **getInNeighbors(G, node):** The function will take a directed graph G and a node as a parameter and will return its in-neighboring nodes.
- **getOutNeighbors(G, node):** The function will take a directed graph G and a node as a parameter and will return its out-neighboring nodes.
- **isNeighbor(G, Node1, Node2):** The function will take a directed graph G , Node1 and Node2 as a parameter and return True if Node 2 is a neighbor of Node 1 , False otherwise.
- **initialize_matrix(rows, cols):** The function will take rows and cols as a parameter and return a matrix (nested list) with all values initialized with -1.
- **adjlst_to_adj_matrix(G):** This function will take a graph G (adjacency list) as a parameter and return the adjacency matrix representation of the

graph G.

- **csv_to_adj_list(file_name)**: This function will take a csv file's name as input which will be in adjacency matrix format and build adjacency list and return it. Please follow the following **tutorial** on how to read and process csv files in Python.

2.1.1 Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_helper_functions.py
```

2.2 Question 1

These functions should be implemented in **q1.py**.

Given the following directed graph, call it DG, whose sets of vertices and edges are given as follows:

Vertices = [1, 2, 3, 4]
(Directed) Edges = [(1, 2), (2, 4), (3, 1), (3, 2), (4, 3), (4, 4)]

Implement the following functions:

1. **create_directed_graph**: Build and return the adjacency list of the given directed graph using appropriate helper functions.
2. **print_graph**: Print the adjacency list representation of the above graph.
3. **in_neighbors**: Compute and return the dictionary where keys are the nodes and values are the list of its in-neighbors.
4. **out_neighbors**: Compute and return the dictionary where keys are the nodes and values are the list of its out-neighbors.
5. **create_adjacency_matrix**: Generates and returns the adjacency matrix representation of the above graph.
6. **check_degree_sums**: Returns whether the sum of the in-degrees of all nodes, the sum of the out-degrees of all nodes and the total number of edges are all equal or not (i.e. True/False)

2.2.1 Testing

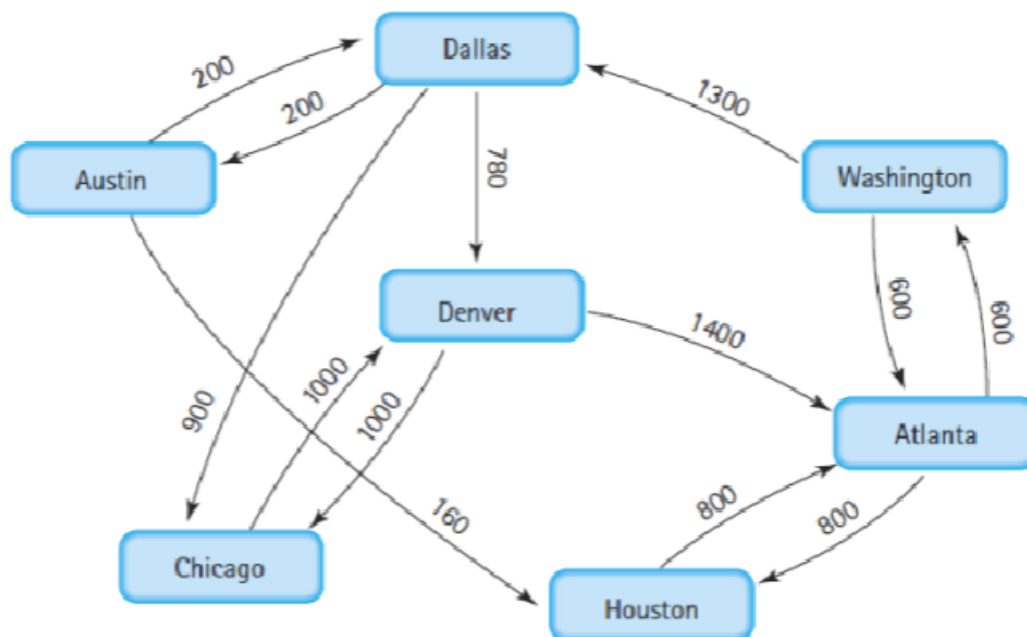
To test your function, type the following command in the terminal:

```
pytest tests/test_q1.py
```

2.3 Question 2

These functions should be implemented in **q2.py**.

The following graph is an example from Rosen (2011). It shows the flights and distances between some of the major airports in the United States.



Implement the following functions:

1. **create_airport_graph**: Build and return the adjacency list representation of the given directed and weighted graph using appropriate helper functions.
2. **max_inbound_outbound_airport**: Compute and return a tuple containing: the airport with the maximum *inbound flights* and the airport with the maximum *outbound flights*.

2.3.1 Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q2.py
```

2.4 Question 3

These functions should be implemented in **q3.py**.

Implement the following functions:

1. **create_cities_graph**: Read CSV file "connections.csv" and build adjacency list.
2. **add_connection**: Recently, A new route was opened from "Balakot" to "Nathiagali". Update your graph with this information and keep its weight as 420.

2.4.1 Testing

To test your function, type the following command in the terminal:

```
pytest tests/test_q3.py
```